ASTITVA

Hackathon 2025

Participant Handbook

Problem Statements · Ground Rules · Tips

Domains	Productivity · Environment · Fintech
Problem Statements	9 total — 3 per domain
Team Size	1 to 4 members
Tech Stack	Open — any language or framework

Format

Teams	1 to 4 members
Tech stack	Any
Submission	GitHub repository with a README
Demo	Live product demo — mandatory, slides alone do not count
Demo duration	3-minute pitch + 2-minute Q&A per team

Ground Rules

✓ Teams of 1 to 4 members	✓ Live demo is mandatory — slides alone do not count
✓ Any tech stack	✓ Final submission via GitHub repo with a README
✓ Open-source libraries and free public APIs are allowed	✓ One submission per team, any domain of your choice
✓ All code must be written during the event	✓ Teams must be present for the judging session
✓ Pre-existing projects or templates are not permitted	✓ Judges' decisions are final

Demo format

Each team gets a total of five minutes in front of the judges.

3 minutes	Live product demo — show the working product, state the problem it solves and walk through the primary user flow
2 minutes	Q&A — judges will ask about a design decision, an edge case or the impact claim you made during the pitch

How you will be judged

Your submission is scored across five areas. Understanding the weights will help you decide where to invest your time.

Pts	Area	What it means for your build
30	**Real-world impact**	Can you name a real person who needs this and explain how their day changes? Start here, before you write a single line of code.
25	**Functionality**	Does the core loop actually work end-to-end? A polished, complete small scope beats a half-built ambitious one every time.
20	**UX and design**	Can someone who has never seen it figure it out in under a minute? Clarity beats beauty.
15	**Originality**	Did you bring a fresh angle? A clever feature or unexpected insight that makes the product unmistakably yours.
10	**Demo and communication**	3-minute pitch + 2-minute Q&A. State the problem, show it live, explain one decision you made and why.

Tips for all teams

- Pick a scope you can finish — a complete small product beats an incomplete large one every time
- Know your user before writing a single line of code — the strongest demos start with a specific, real story
- Prepare your demo data in advance — live-typing dummy data during a 3-minute demo wastes precious time
- Show an edge case or error state — teams that demo graceful failure look significantly more credible
- The README matters — clear setup instructions tell judges the team thinks about users beyond themselves

DOMAIN 1 · PRODUCTIVITY

This domain covers the invisible friction in daily knowledge work — how we learn, how we stay on track and how we keep each other accountable. Every problem statement targets a gap that students, freelancers and professionals hit repeatedly, often without realising it has a name.

PS 1.1 ·

The Knowledge Decay Problem

Build a personal knowledge retention system that fights the Ebbinghaus forgetting curve

Background

Professionals and students consume enormous amounts of content every day — articles, videos, documentation, lectures, podcasts and books. Studies show that within 7 days, people forget approximately 90% of what they consumed (Ebbinghaus Forgetting Curve). Bookmarking tools save links but not knowledge. Notes apps become graveyards of half-finished thoughts. The gap between 'I read something useful once' and 'I can recall and apply it when I need it' is enormous and growing.

The Problem in Detail

- Students highlight entire textbook chapters but cannot recall the key insight three days later
- Professionals save dozens of articles weekly and revisit fewer than 5% of them
- Existing flashcard tools require significant setup effort and feel like schoolwork rather than a daily habit
- Notes apps become overwhelming — too much structure, not enough retrieval
- There is no feedback loop telling you what you actually retained versus what you think you retained

The Challenge

Build a lightweight, frictionless system that captures the single most important insight from anything a user consumes, and resurfaces it at the scientifically optimal moment — making retention feel almost effortless.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Capture screen — user enters a single bite-sized takeaway (the core insight, not a full note), tags it with a topic and optionally records a source name or URL
2	Spaced repetition engine — each takeaway is scheduled for review at Day 1, Day 3, Day 7, Day 14 and Day 30 intervals based on the Ebbinghaus forgetting curve
3	Flashcard review session — present due takeaways one at a time; user marks each as Remembered (interval advances) or Forgot (interval resets to Day 1)
4	Knowledge map — a visual overview of all topics the user has captured knowledge in, with a strength indicator per topic based on cumulative retention scores
5	Search and retrieve — user can search all past takeaways by keyword or topic to find relevant knowledge while working on a real task

6	Retention score dashboard — a running percentage of takeaways successfully recalled over time, broken down by topic

PS 1.2 ·

The Context-Switch Tax

Build a personal work dashboard that acts as mission control for a user's active tasks

Background

Remote and hybrid workers lose an estimated 23 minutes of productive focus every time they switch between tasks. On a typical day, people juggle 5 to 10 different tools — email, Slack, documents, calendars, project boards and code editors — and there is no unified place to track what they were doing, where they left off and what needs attention next. Every time they return to a paused task, they spend minutes reconstructing context that should have been saved automatically.

The Problem in Detail

- Switching from a deep coding session to a meeting and back costs significant ramp-up time
- Browser tabs and sticky notes are the dominant context-saving strategy — both fail at any real scale
- Most task managers track what to do but not the live context of work actually in progress
- Returning to a paused task after hours or days often means re-reading entire threads from scratch
- There is no objective record of how time was actually distributed — only how it was planned

The Challenge

Build a personal mission control that captures, preserves and restores the complete context of a user's active work — so that resuming a paused task takes seconds, not minutes.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Session creation — user creates a work session tied to a specific task or project with a title, optional notes, a list of relevant links and a live timer that starts automatically
2	Pause and resume — user can pause an active session (timer stops, state is fully preserved) and resume it later; the app shows a clear indicator of which session is currently active
3	Session state — each session persists notes-in-progress, checklist items with individual completion states and any links the user added during the session
4	Resume packet — when a user resumes a paused session, they are shown a summary of what was being worked on, which checklist items are pending and the relevant links
5	Daily timeline — a chronological view showing how time was actually distributed across all sessions throughout the day, with duration per session and total focused time

6	Session archive — completed and paused sessions are stored and searchable so users can reference the context of past work

PS 1.3

Meeting Accountability Gap

Build a meeting outcome tracker focused purely on decisions and action items

Background

Teams hold an average of 11 to 15 meetings per week. Action items are discussed, nodded at and promptly forgotten. Studies show that within 48 hours, most participants have forgotten 50 to 70 percent of what was decided. Meetings repeat the same conversations. Decisions get re-debated. The person who was supposed to do something never received a written record of the commitment. Accountability evaporates — not because people are irresponsible, but because there is no lightweight system to capture it.

The Problem in Detail

- Meeting notes are either not taken, or taken in a personal notebook no one else can see
- Action items discussed verbally are rarely assigned a named owner and a concrete deadline
- Follow-up emails summarising meetings are inconsistently written and quickly buried in inboxes
- There is no single place where a person can see all the tasks they owe across multiple meetings
- Overdue action items have no visible consequence — they silently age until someone asks about them

The Challenge

Build a focused meeting outcome tracker — not a transcription tool, not a scheduler — that captures decisions and action items in real time and keeps every participant accountable to their commitments.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Meeting log — user creates a meeting record with title, date, a list of attendees and a brief agenda or context note
2	Decisions log — user adds key decisions made during the meeting (what was decided, not who said what); decisions are displayed prominently
3	Action items — user adds action items with a title, an assigned owner selected from attendees, a deadline date and a status: open, in progress or done
4	Personal feed — each attendee has a view showing only their action items across all meetings, sorted by deadline, with overdue items highlighted
5	Staleness indicator — any action item past its deadline and not marked done is flagged visually and surfaced prominently in both the meeting view and the personal feed

6	Weekly digest — a summary view showing all decisions made this week, all action items due this week, overdue items and items completed this week

DOMAIN 2 · ENVIRONMENT

This domain tackles the gap between wanting to live sustainably and having the data to act on it. Every problem statement is entirely formula and data-driven — no external dependencies required. These are among the most accessible problem statements in the hackathon and are well-suited to teams working with frontend technologies only.

PS 2.1 ·

The Phantom Load — Home Energy Waste Finder

Build a home energy audit tool that reveals hidden electricity costs appliance by appliance

Background

Households waste 5 to 10 percent of their total electricity on phantom loads — devices that draw power even when turned off or on standby. Chargers, gaming consoles, smart TVs, desktop monitors and set-top boxes silently consume electricity around the clock. Beyond phantom loads, most people have no idea which appliances are costing them the most money or generating the most carbon. Electricity bills show a single number at the end of the month but offer zero behavioural insight.

The Problem in Detail

- Most people cannot name which three appliances in their home consume the most electricity
- Standby power consumption is invisible — there is no meter on each individual appliance
- Electricity costs remain abstract until broken down per appliance, per day and per year
- Carbon footprint of home energy use is rarely considered in daily decisions about appliance usage
- Upgrade decisions such as switching to LED or buying a 5-star AC are made without knowing the actual payback period

The Challenge

Build a home energy audit tool that lets anyone build a virtual inventory of their appliances, calculates exactly how much each one costs in money and carbon, and shows the most impactful changes they can make.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Appliance inventory — user builds a room-by-room list selecting from a pre-built catalog with average wattage values, or entering custom wattage for unlisted devices
2	Usage input — for each appliance, user inputs estimated daily active usage hours and whether the device stays on standby when not in active use
3	Per-appliance calculations — daily kWh, monthly kWh, yearly kWh, cost in ₹ (user inputs their per-unit rate) and CO_2 footprint using a regional emission factor dataset
4	Waste score ranking — appliances are ranked by a composite score combining cost, carbon output and standby usage efficiency; the top offenders are clearly highlighted
5	Swap and save recommendations — for the highest-waste appliances, generate specific quantified recommendations (e.g. 'Replacing your 60W incandescent bulbs with 9W LEDs saves ₹2,400/year and 85 kg CO_2')

6	What-if simulator — user models scenarios: what if I reduce AC usage by 1 hour per day? What if I unplug my TV on standby? The app shows projected annual savings in ₹ and CO_2

PS 2.2 ·

Campus Carbon Tracker

Map your college's collective environmental footprint through the daily habits of its students

Background

India is the third-largest emitter of greenhouse gases globally, and per-capita emissions from urban educated populations are rising. College campuses are microcosms of this trend: thousands of students making daily choices about commuting, eating, printing and energy use, with almost no visibility into the cumulative impact. Sustainability campaigns on campuses are common; actionable, data-driven personal feedback is rare.

The Problem in Detail

- Students know climate change is important but have no sense of their own numerical contribution to it
- Campus sustainability initiatives lack data — they cannot measure whether behaviour is actually changing
- Generic carbon footprint calculators online are not campus-specific and have no social component
- Peer comparison and friendly competition are known to drive behavioural change — no campus tool uses this
- Weekly habits like commute mode and diet have a far larger carbon impact than most students realise

The Challenge

Build a campus-specific carbon tracker that turns individual daily habits into a personal footprint score, aggregates these into a live campus-wide picture, and uses social comparison and weekly challenges to drive genuine behaviour change.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Daily habit log — user logs habits across four categories: commute mode (walk, cycle, two-wheeler, car, bus, auto), meals (vegetarian, chicken, beef, mixed), paper and printing usage and estimated personal electricity use on campus
2	Personal footprint calculation — convert logged habits into a daily carbon footprint in kg CO_2 using standard emission factors; show a running weekly and monthly total
3	Campus leaderboard — aggregate all users' footprints into a live campus-wide total and per-capita average; display a leaderboard of lowest-footprint users on an opt-in basis
4	Comparison view — show the user how their footprint compares to their college average, a national average and a global average with a clear visual representation

5	Weekly challenges — one weekly challenge is offered (e.g. 'Go meatless twice', 'Walk to class five days'); users opt in and the app tracks completion against their logs
6	Habit insights — show the user which of their habits contributes most to their footprint and the single highest-impact change they could make this week

PS 2.3 ·

The Water Blindspot

Build a personal and household water footprint calculator that makes invisible consumption visible

Background

Water scarcity affects over 2 billion people globally, and India faces some of the most acute water stress in the world. Yet most individuals have no real understanding of how much water they actually consume — both directly through daily use and indirectly through the food they eat. Water bills show a volume number but provide no insight into which behaviours are driving it or how usage compares to any meaningful benchmark.

The Problem in Detail

- People know they should conserve water but cannot quantify the impact of any specific behaviour change
- Indirect water — the water embedded in food production — is almost entirely invisible to consumers
- 1 kg of beef requires approximately 15,400 litres of water; almost no one is aware of this figure
- Water bills arrive monthly and are forgotten within days — there is no continuous feedback loop
- There is no tool that bridges direct water usage with dietary choices in a single coherent view

The Challenge

Build a water footprint tracker that combines direct household water usage with the indirect water embedded in dietary choices, shows users how they compare to regional and global benchmarks, and gives them clear, quantified actions they can take.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Direct water logging — user logs daily usage with smart defaults: shower (duration × litres per minute), dishwashing (hand or machine), laundry (number of loads), garden watering and other custom uses
2	Indirect water calculator — user logs dietary choices; the app calculates virtual water using a reference dataset (beef ≈ 15,400 L/kg, chicken ≈ 4,300 L/kg, rice ≈ 1,670 L/kg, vegetables ≈ 200 L/kg)
3	Footprint dashboard — today's total footprint in litres split between direct and indirect consumption, with a clear visual breakdown
4	Benchmark comparison — compare the user's daily and weekly footprint against regional average, national average and global average; show clearly whether they are above or below

5	What-if simulator — 'If I switch from beef to chicken three times a week I save X litres per month'; 'If I cut my shower by 2 minutes I save Y litres per week'
6	Trend visualisation — a weekly and monthly chart showing the user's footprint over time so they can see whether habits are improving

DOMAIN 3 · FINTECH

This domain addresses the gap between what people think they earn or spend, and what is actually happening with their money. Each problem targets a familiar financial frustration that most college students and young professionals have experienced personally.

PS 3.1 ·

The Freelancer's Blind Spot

Build a project profitability tracker that reveals whether freelance work is actually worth it

Background

Freelancers and gig workers often have no visibility into how profitable their projects actually are. They track hours and invoice amounts, but not the hidden costs — the back-and-forth emails, the revision rounds, the scope creep conversations, the unpaid admin work and the client calls that were never in the original quote. Many freelancers discover only retrospectively that a project appearing to pay ₹50,000 was actually paying less than minimum wage once all real time was accounted for.

The Problem in Detail

- Freelancers track billable hours but routinely under-count communication and revision overhead
- Scope creep happens gradually — each individual addition seems minor but cumulatively destroys the margin
- There is no live signal warning that a project's effective rate is dropping below a sustainable floor
- Portfolio-level analysis is almost never done — freelancers cannot say which client types are actually profitable
- Standard invoicing tools focus on what to bill, not whether the economics of the project make sense

The Challenge

Build a project profitability tracker that calculates the true effective hourly rate on every project in real time — including all non-billable time — and helps freelancers identify which work is genuinely worth taking on.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Project setup — user creates a project with a title, client name, project type, a total fixed price or hourly rate and an estimated total hours budget
2	Time logging — user logs time sessions tagged as billable (direct work) or non-billable (admin, emails, calls, revisions); a live timer is available for real-time tracking
3	Live effective rate — the app calculates and prominently displays the current effective hourly rate: total project value ÷ total hours logged (billable + non-billable combined); updates in real time
4	Threshold alert — user sets a personal minimum effective rate floor (e.g. ₹500/hour); the app flags clearly when any project's current rate drops below this threshold

5	Non-billable breakdown — a visual breakdown of where non-billable hours are going (communication, revisions, admin, scope additions) so the user can identify patterns
6	Portfolio view — across all projects, show total earned, average effective rate, most and least profitable projects and a breakdown by project type or client

PS 3.2 ·

The Subscription Black Hole

Build a subscription tracker that tells users not just what they pay, but what they actually get

Background

The average person pays for 8 to 12 subscriptions simultaneously and actively uses fewer than half of them. Streaming services, productivity tools, gym memberships, cloud storage, news subscriptions and app premium tiers — each one is individually small enough to approve without scrutiny. Collectively they can represent ₹2,000 to ₹4,000 per month of largely unnoticed spending. Cancellation is deliberately made difficult. Bank statements list the charges but give no indication of whether the user received any value.

The Problem in Detail

- Most people genuinely do not know how many subscriptions they are paying for at any given time
- Renewal dates are invisible until the charge appears on a bank statement
- There is no record of whether a subscription was actually used during the period it was charged for
- The mental overhead of managing and cancelling subscriptions is high enough to prevent action
- Nobody does the compounding math on what subscription spending could become if redirected instead

The Challenge

Build a subscription tracker that gives users complete visibility into their recurring spending, helps them evaluate each subscription's actual value, and shows them concretely what cancelling underused services could mean for their finances.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Subscription registry — user adds each subscription with: name, monthly or annual cost, billing cycle, renewal date, category (entertainment, productivity, health, utilities, other) and payment method
2	Monthly value rating — once per billing period, user rates each subscription: Used frequently, Used occasionally or Never used; ratings are stored as a history
3	Spending dashboard — total monthly and annual burn broken down by category with a visual chart; the most expensive and least-used subscriptions are highlighted prominently
4	Cancel candidates — subscriptions rated Never used or Used occasionally for three or more consecutive billing periods are automatically flagged with a clear visual indicator

5	Renewal calendar — a 30-day view showing upcoming renewal charges with amount and service name; filterable by 7, 14 or 30 days ahead
6	Savings projection — for flagged subscriptions, show total annual saving if all are cancelled and what that money could become in 5 and 10 years with compound interest at a conservative rate

PS 3.3 ·

Split It Fair

Build a group expense manager that handles the real-world complexity of shared spending

Background

Splitting expenses among friends, roommates and travel groups is one of the most consistently friction-filled aspects of shared life. Simple apps handle 50/50 splits, but real-world shared spending is messier: groceries where one person is vegetarian, utilities split by room size, a group trip where one person fronted the hotel costs, recurring rent that needs tracking month after month. The person who always pays first builds quiet resentment. The person who always forgets to pay causes it.

The Problem in Detail

- Equal splits are rarely fair — people have different diets, different room sizes and different participation levels
- Multi-currency trips create a calculation burden most people handle with rough approximations
- Settlement requests pile up and the mental model of who owes whom across a group becomes genuinely confusing
- Recurring costs like rent and utilities are tedious to re-enter every month
- There is no clear, neutral record that both parties can refer to when a disagreement arises

The Challenge

Build a group expense manager that handles flexible real-world splitting scenarios, automatically calculates optimised settlements, and gives every member of a group a clear and unambiguous view of their position.

What to Build

Your submission must address the following functional areas. A complete, well-executed subset is better than a broken full implementation.

1	Group management — user creates a group, adds members by name and can manage multiple groups (e.g. 'Goa Trip', 'Flat 4B', 'Office Lunch Club')
2	Expense logging — user adds an expense with: title, total amount, currency, who paid, date and a split method: equal, percentage-based, share-based (e.g. 2x for the larger room) or item-level
3	Multi-currency support — expenses in different currencies are converted to a base currency using a free forex API; the conversion rate used is stored with each expense for transparency
4	Debt simplification — the app calculates the minimum number of transactions needed to settle all debts; rather than A paying B and B paying C, A pays C directly
5	Member dashboard — each member has a personal view showing: total they owe, total owed to them and a list of specific amounts owed to or from each individual in the group

6	Recurring expenses — user can mark an expense as recurring monthly; the app generates the recurring entry automatically each period

Build something people actually need. Good luck.

Astitva Hackathon 2025 · Participant Handbook